EDUCATION LABS TAKE-HOME · OPTION B

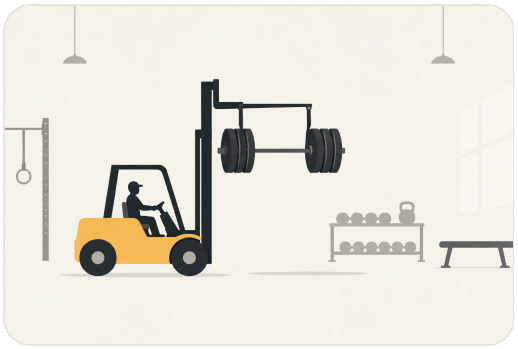
Learn mode

Claude does the work. You learn from it while it happens.

Arvind · ant-takehome.vercel.app · github.com/arvindm1991/ant-takehome

The problem

> "When AI agents handle complex tasks autonomously, humans can become passive observers rather than active learners, missing opportunities to develop their own skills and understanding."
>
> The brief, challenge 2: Cognitive engagement





That's the problem I took on. Agents are a forklift in the gym and a microwave in the kitchen: the work gets done, but the muscle doesn't grow, the knives go dull and the pots rust. Showing more of the work doesn't fix it. Claude already shows its reasoning and code, and people skip to the answer.

The good news: unlike a course, this isn't learning for its own sake. The task is real, the person just asked for it, and an expert is doing it right now. It's on-the-job training, where you learn the job while a skilled hand does it.

> The goal: add back the right amount of effort for the learner while the work gets done, without slowing the work down.

Why Option B

Option A is customer education: teaching people Claude's features. Option B is general education: helping people get better at their own work, whether that's programming, writing or analysis. B is the bigger opportunity. Finding features matters, but people losing their own skills is the bigger risk, and it grows as agents take on more of the work.

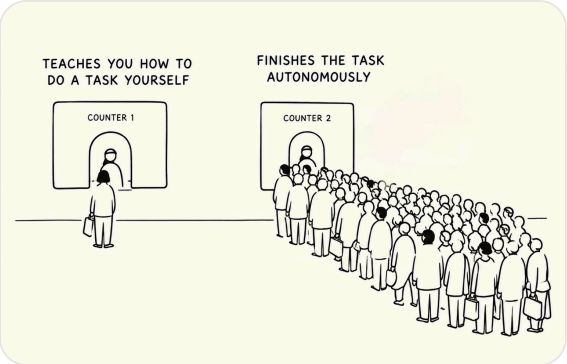
It was also a personal choice. I trained as an aerospace engineer, then built an award-winning edtech company and have stayed in education since. I care about building learning that is pedagogically sound and realistic for the next generation of learners, and I was already working on ideas in this space. Option B was an easy pick.

What I learned from people who use Claude every day

Before building, I talked to three developer friends who use Claude daily. I didn't ask whether they'd use a learning tool; people say yes to that and then don't. I asked what they actually do while the agent is building. All three switch to other work and stop watching. I also asked how they'd feel about being quizzed inside Claude.

Those conversations shaped the product more than anything else. Learning has to meet people in the few moments they're already looking, stay out of the way the rest of the time, and never feel like a test they didn't ask for. I used what I learned from them, along with nearly a decade of building in education, to set the direction of the prototype.

Why inside Claude, not a separate app



When something needs doing, people go where it gets done fastest. That's why so many good learning products get little use; I've built some of them. A separate learning app that makes you work for the answer is counter 1: good for the few who choose it, empty for everyone else.

So I didn't build counter 1. **Learn mode puts the teaching at counter 2**, inside the Claude session where people already are, and it never makes the task wait.

What I built

Learn mode lives inside Claude. When it's on, a learning agent runs alongside Claude. Claude does the task exactly as it would anyway; the learning agent watches Claude's work and teaches you from it.

Three ways in. A switch that's always there in the top bar; a suggestion in the moment, under a task worth learning from; and spaced review, which brings you back to something you learned in an earlier session, through the Inbox or inside a related task. The third is a retention and value loop: it gives people a reason to return, like a streak, and every return adds value, because spaced repetition is one of the most effective ways to make learning stick. The prototype has all three; which mix works best, and for whom, is something to A/B test.

Out of the way, but not silent. Learn mode never pauses Claude or covers your work. When it's off, it offers one small suggestion under a task, which you can wave away with ×; once dismissed, it doesn't come back for that task. When you turn it on, it's proactive: it starts teaching as soon as there's something worth learning, and checks in when Claude writes a step you made a prediction about.

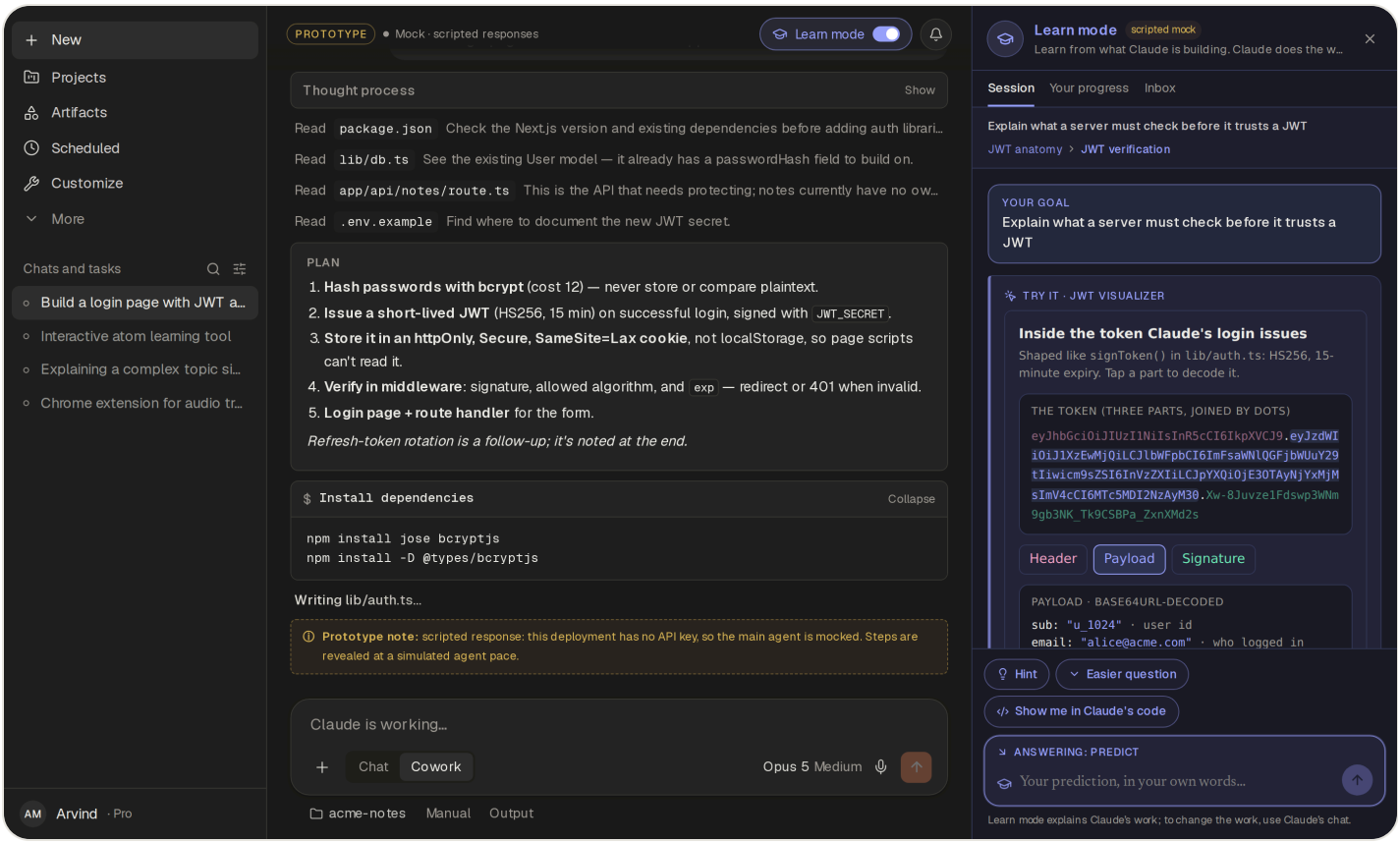
Claude (main agent)
Does the task at full quality. Doesn't know Learn mode exists; nothing about the learner enters its context.

→ read-only

Learn mode (learning agent)
Reads Claude's plan, files and code as they appear, and teaches from them.

EXAMPLE: ADDING LOGIN TO A NOTES APP

1. With Learn mode on, you ask Claude to build a login page with JWT auth for your notes app. Claude starts reading the repo.
2. Learn mode lists a few things worth learning from this task, such as how the server checks a login token, or why passwords are stored with bcrypt. You pick one.
3. It shows you the kind of token Claude's code will create, split into its three parts. You tap each part to see what's inside.
4. It asks what `verifyToken()` needs to check. Claude hasn't written that function yet. When it does, Learn mode compares your answer with Claude's code and links to the lines.
5. If you missed something, you can ask for a hint, see a demo, or jump to the relevant code. If you got it right, you can go a level deeper or move to a related topic.
6. After a few questions it sums up what you covered. A few days later, or when you work on something related, it asks you one question about it again.



Left: Claude is writing `lib/auth.ts` and isn't waiting on anything. Right: Learn mode has read Claude's reasoning and plan, built a visualizer of the token that file will issue, and asks you to predict what `verifyToken()` must check before Claude writes it.

How the learning agent works

It's a real agent. It chooses its own actions with tools, remembers the learner across sessions, and works toward the goal the learner picked.

WHAT IT SEES EACH TURN

- What you asked Claude, and Claude's steps so far: plan, files read, code written. Steps Claude hasn't shown yet appear by name only, so it can ask you to predict them without giving them away.
- What it knows about you: how well you know each topic, mistakes you keep making, your last few answers.
- Your goal and the conversation so far.
- A one-line teaching approach chosen by code from your level. For example: new to this, so show it first; missed twice, so show it and then ask something easier.

WHAT IT CAN DO

These are the tools built for the prototype, not the full set the agent could have. The aim was a solid product surface with a low floor and a high ceiling: easy for anyone to start with, room to add richer tools, and general enough to work across subjects.

TOOL	WHAT IT'S FOR
Suggest goals	4–5 things worth learning from this task, from where to look in the repo to a stretch goal.
Ask a question	Predict the next step, explain it back, “what if…”, or pick the files you'd open. It writes the answer key from Claude's code.
Build an interactive	Its preferred way to teach. A separate Opus call builds it from Claude's code and runs it in a sandbox with no network access.
Hint · Explain · Wrap up	A nudge that never gives the answer; a short explanation tied to Claude's code; a recap when the goal is done.

Everything it produces has to point at the step in Claude's work it's about; references to steps that don't exist are dropped. Two helpers sit beside it: a grader (Sonnet) that scores written answers against the answer key and names the misconception, and a quick check (Haiku) that decides whether a task is worth learning from at all.

WHEN IT ACTS

It responds to events: a task starts, you pick a goal, you answer, you pick a next step or type a question, or Claude writes a step you made a prediction about. It keeps going on its own where that helps. After you pick the files you'd open, it moves to the next question while Claude is still reading, then checks your picks against the files Claude actually opened. Once you've answered three questions on a goal, it wraps up with a recap.

GETTING AHEAD OF CLAUDE

Claude never waits for Learn mode and isn't influenced by it. Learn mode gets ahead by reading Claude's reasoning as it streams and the plan Claude makes before it writes code. That's how it can ask you to predict `verifyToken()` before the file exists. In a real integration it could start even earlier, as soon as the main agent's own reasoning is available.

How it remembers what you know

- **The model grades, code keeps score.** The grader says right, partly right or wrong. Plain code turns that into a score, so scores are consistent and can be checked.

- **A score per topic** starts at 30% and moves toward each new result. Predictions and “what if” answers move it more than recall. An answer given after a hint counts for less.
- **Every attempt is kept as evidence:** the question, your answer, the result, and which step of Claude's work it was about. The agent sees your recent attempts, and the recap shows before and after.
- **Misconceptions stay open** (for example, “the token's contents are encrypted”) until you later get that topic right without a hint. The agent aims questions at them.
- **Spaced review.** A topic comes back after 1, 3, 7, 16 and 35 days. A right answer pushes it further out; a miss brings it back tomorrow. Due topics under 70% show up as refreshers.
- **Related work brings topics back.** When a new task touches something you learned, you get a one-question refresher there, or a question that connects the two (bcrypt and rate limiting, say).
- **Where it lives:** in the browser for the prototype. The same shape would move to Claude's memory.

How I got here

- **The first idea everyone had was wrong.** Every model I brainstormed with, Claude included, first suggested pausing the agent at key decisions and letting the user choose. That makes the work only as good as the learner, and fills the agent's context with guesses. So the learner rides alongside instead of steering.
- **Spec before code.** A numbered log of decisions and six milestones, each tested before the next.
- **Real Claude API from the start.** Opus for the main agent (reasoning streamed live), Sonnet for the learning agent and grader, Haiku for the “worth learning?” check, and Opus for interactives.
- **Changed it after using it.** A blank chat box felt like “a second Claude”, so it became goals and next steps. That felt like homework, so now it shows before it asks, has one text box of its own, and starts by itself when Learn mode is on. Then I made it work on phones.
- **Safe for a public link.** Rate limits, size limits, and prompts that treat Claude's output and user text as material, not instructions.

Keeping people in charge

- **Claude still does the task, fully.** Learning never blocks it, slows it or changes it.
- **You choose** whether to learn, what, and how deep. Everything can be skipped, and Learn mode is one switch.
- **Understand before you review.** Knowing why Claude made each choice is what lets you check its work and push back.
- **It never does the task for you.** It shows, asks and checks. It asks about Claude's choices rather than judging them, because it doesn't have Claude's full context.

Learning principles

PRINCIPLE	WHERE IT SHOWS UP
Learn on the job	An expert does the real task while the learning agent explains, asks and checks.
Learn when it matters	Topics come from the task you just asked for, so there's no need to convince you they matter.

PRINCIPLE	WHERE IT SHOWS UP
Guess, then check	You predict before the code exists, then compare with what Claude wrote.
Do, don't just watch	Interactives you play with instead of text you read. A live map of Claude's work was dropped: it feels like teaching without showing anyone learned.
Support, then step back	Hints, never answers; an easier question or an interactive after a miss.
Space it out and mix it up	Refreshers at growing gaps, and when a related task comes up.
Know the learner	Scores, evidence and misconceptions decide what comes next and skip what you already know.

Measuring success

- **Main measure:** how often people get a refresher right on the first try, days later and in a different task. It shows what stuck, not how long anyone spent.
- **Progress:** topic scores over time, and misconceptions cleared.
- **Adoption:** offered → opened → first answer → goal finished, compared for the switch and the in-chat suggestion.
- **Guardrails:** no change to how long tasks take or how happy people are with them; how often people switch it off; how often people type a request for Claude into Learn mode's box.
- **Checks on the learning agent:** it uses Claude's actual code, doesn't give away answers, grades like a human would, and shows before it tells.

Scaling to millions

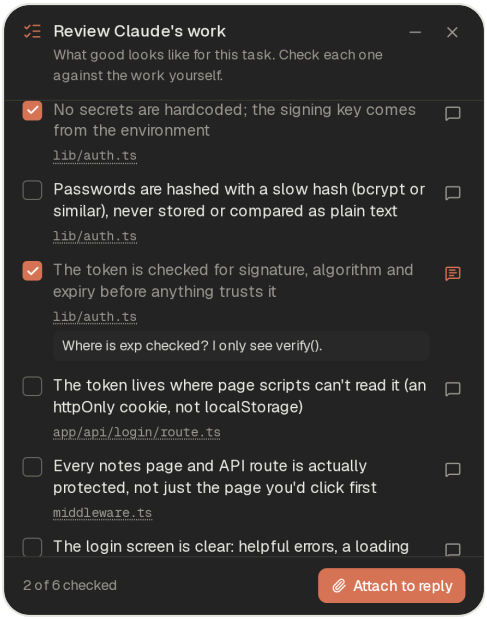
- **No new app to find:** it's a switch in Claude and Claude Code.
- **Works beyond code:** topics come from the work itself, not a hand-written course. The same loop fits writing and analysis.
- **Different defaults for different people:** on by default for students (set by schools); off for professionals, with occasional suggestions; stretch goals and review for experienced users.
- **Cost follows use:** it only runs when on; Sonnet and Haiku do the watching and deciding; Opus builds interactives only when someone asks to see or try something.
- **Rollout:** behind a flag, with each change below run as an A/B test and kept only if it moves the main measure without hurting the guardrails.

A second pattern: reviewing Claude's work

Education Labs studies AI fluency through the behaviors people show when working with AI, some visible in a conversation and some not. Those behaviors are a good starting point, but it's easy to design to the test. Suggested follow-up questions are an example: picking one looks like iteration, but often the model is choosing where the task goes and the person makes a token choice. What matters is whether the person is steering or just letting the task expand.

So I prototyped a pattern that asks for the person's own judgment. When Claude finishes work, a review checklist opens with what good looks like for this task, written from the goal and the files. For the login page: no secrets are hardcoded, passwords use a slow hash, the token is checked for signature and expiry, the login screen fits the app. Nothing is pre-checked and nothing says whether Claude passed. You tick items as you check them, comment on any of them, and attach the checklist to your reply, so what you found steers Claude's next turn. It can be minimized or closed.

It prompts review of the work like nothing else does, and it shows the difference between reviewing and rubber-stamping: everything ticked in a few seconds with no comments reads as a token choice. It's separate from Learn mode, and we'd A/B test it against suggested follow-ups and against nothing.



What we'd test, and what comes next

The prototype is a starting point, not a final answer. These are the questions we'd settle with A/B tests, each judged on the main measure (first-try refresher accuracy days later) with the same guardrails: tasks take no longer, people are no less happy with them, and switch-off and dismissal rates stay low.

QUESTION	WHAT WE'D COMPARE
How people find it	An always-there switch, a suggestion in the moment under a task, or Learn mode as a mode in the model picker. Also the suggestion's wording, whether it shows while Claude works or after, and how refreshers bring people back.
Who has it on by default	On for students (set by schools), off with suggestions for professionals, and how many suggestions a day before it's too many.
How often it speaks up	Check-ins only when Claude writes a step you predicted, or also on other new files. Refreshers in the Inbox only, or also inside related tasks.
Where the refresher badge lives	On the bell in Claude's top bar, or only in Learn mode's Inbox if the badge proves noisy. Instrumented, like every test here.
Show first or ask first	Opening with an interactive and then a question, against a question first. Interactives cost more, so they have to earn it.
How long a session is	Wrapping up after three questions on a goal, against shorter and longer sessions.
Doing, or following along	The active loop against a live map of Claude's work, which was parked because it feels like teaching without evidence of learning. It may still suit people who want to understand the work but not answer questions.

WHAT COMES NEXT

- **Start even earlier:** in a real integration, Learn mode can read the main agent's own reasoning as it happens, instead of the summary the prototype streams.
- **Claude Code:** the same agent in the terminal, using the minutes while the agent works.
- **Beyond code:** writing and analysis tasks run through the same loop; each needs its own interactives and a check that its questions are grounded.
- **Memory that follows you:** move the learner model from the browser into Claude's memory, across devices and products.
- **When Claude gets something wrong:** measure first with seeded mistakes, and keep asking rather than judging, since Learn mode lacks Claude's full context.
- **Evals and analytics:** the eval suites and the events to log are written up in the repo; running them is the first step before any wider test.
- **More tools:** the agent's tool set is deliberately small; new ways to teach plug into the same loop.

Simulated in the prototype: the repository, the pacing of Claude's steps, and a “+3 days” clock for spaced review. A scripted fallback runs if API credits run out.